

Architettura dei Calcolatori

Secondo Appello – 25 giugno 2024 – 1h45

1. Implementare in assembly RISC-V, la due funzioni C **sum_vec** e **square**.

```
short int vet_sum(short int A[], short int B[], short int C[], int N)
{C[i]=A[i],+B[i]+square(i)}
```

Assumendo che la funzione square(i) fa il quadrato di i. Gestire correttamente lo stack come da calling convention, così come il passaggio dei parametri di ingresso e di uscita sui registri appropriati. Non serve scrivere la funzione main che invoca sum_vec.

2. Dato il seguente snapshot della memoria istruzioni, contenente un frammento di codice binario RISC-V (Quello sotto è un esempio di codice assembly, al esame non cera questo ma uno simile)

```
0x4000 0x00A904B3
0x4004 0x00B48023
0x4008 0x00040A67
0x400C 0x00190913
0x4010 0xFF1FF06F
0x4014 0x00008067
```

Si decodifichino le istruzioni nell'equivalente codice assembly.

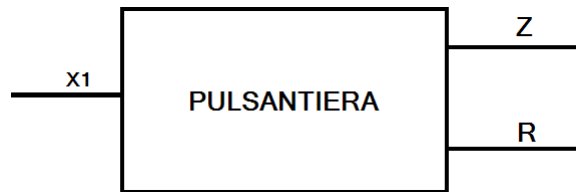
Dove sono implicate istruzioni di alterazione del control flow si trasformino gli offset di salto in opportune label.

All'esame vero il codice tradotto era simile a queste con riportate i registri x10,x11, saranno usati nel esercizio 5

```
L1:
    bge x10,x6, 0x020
    slli x7,x7,3 # i*8
    Addi x7,x7,x10 #a[i]
    lh x11. 0(x7)
    addi x11,x11,x0 #a[i]=0
    sh x11. 0(x7)
    Jalr ... 0xFFFF5 (L1)
```

3. Si progetti una rete logica sequenziale con un segnale di ingresso X a 1 bit e due segnali di uscita a 1 bit:

- Z che riconosce la stringa binaria 0110
- R che riconosce la stringa binaria 0101



- Determinare il numero minimo di stati necessari per risolvere il problema, scegliendo Mealy o Moore come si ritiene più opportuno. Disegnare il diagramma degli stati.
- Suggerire una codifica per gli stati e derivare la tabella di verità. Da lì popolare le mappe di Karnaugh e ricavare le forme minime per le reti di stato futuro e per le uscite.
- Disegnare il circuito finale.

4. Si consideri di eseguire il seguente stralcio di codice su una pipeline RISC-V con logica di forwarding e con dynamic branch predictor a **due bit** inizialmente settato a BRANCH TAKEN

```
li t2,3 // UB
li t0,0 // i = 0
loop_i
    li t1,0 // j = 0
loop_j
    addi t1,t1,1
    blt t1,t2,loop_j
    addi t0,t0,1
    blt t0,t2,loop_i
jr ra
```

- Si calcoli la misprediction rate.
- Sapendo che:
 - per ogni predizione corretta un'istruzione di tipo branch impiega 2 cicli ad eseguire, mentre in caso di misprediction ne impiega 8
 - le istruzioni aritmetico-logiche impiegano 2 cicli a eseguire
 - il salto incondizionato impiega 8 cicli ad eseguire

Si calcoli il CPI medio per l'esecuzione del programma.

5. Si modifichi il programma assembly dell'esercizio 2. considerando i valori iniziali:

- a0= 0x0040000
- a1= 0x0040020
- a2= 16

Si immagini di eseguire il programma così modificato su una CPU RISC-V dotata di **data cache** 2-way associative con 2 bit di INDEX e 4 bit di OFFSET.

Qual è il miss rate risultante per la cache istruzioni?

